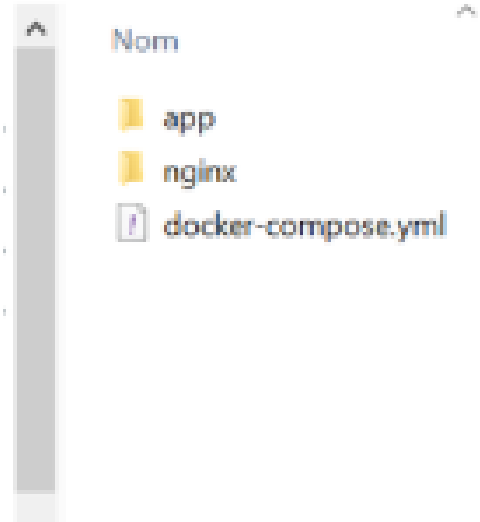
TP 3 – Architecture distribuée

EL KHAYAR Youssef

1 Structure du projet & prérequis



J'ai créé les dossiers pour bien séparer chaque composant. C'est plus propre et ça facilite la maintenance.

2 Docker Compose — Lancer tous les composants

docker-compose.yml

C: > Users > pc > tp-archi-distribuee > docker-compose

```yaml
version: '3.8'
Run All Services
services:
  Run Service
  mongodb:
    image: mongo:6
    container_name: mongodb
    ports:
      - '27017:27017'

  Run Service
  redis:
    image: redis:7-alpine
    container_name: redis
    ports:
      - '6379:6379'

  Run Service
  rabbitmq:
    image: rabbitmq:3-management
    container_name: rabbitmq
    ports:
      - '5672:5672'
      - '15672:15672'
    environment:
      RABBITMQ_DEFAULT_USER: admin
      RABBITMQ_DEFAULT_PASS: admin
```

C: > Users > pc > tp-archi-distribuee > docker-compose.yml

```yaml
      PORT: 3001
      INSTANCE: 'Instance-1'
    depends_on:
      - mongodb
      - redis
      - rabbitmq

  ▷Run Service
  app2:
    build: ./app
    container_name: app2
    environment:
      PORT: 3002
      INSTANCE: 'Instance-2'
    depends_on:
      - mongodb
      - redis
      - rabbitmq

  ▷Run Service
  nginx:
    image: nginx:alpine
    container_name: nginx
    ports:
      - '80:80'
    volumes:
      - ./nginx/nginx.conf:/etc/nginx/nginx.
    depends_on:
      - app1
      - app2
```

J'utilise un seul fichier pour lancer tous les services en même temps. Je définis pour chaque conteneur son rôle et ses dépendances.

3 Configuration Nginx — Load Balancer

nginx.conf

C: > Users > pc > tp-archi-distribuee > nginx > nginx.conf

```
events {
    worker_connections 1024;
}

http {
    upstream backend {
        # Définition des deux instances applicatives
        server app1:3001;
        server app2:3002;
    }

    server {
        listen 80;

        location / {
            proxy_pass http://backend;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
        }
    }
}
```

J'ai configuré Nginx pour qu'il écoute sur le port 80 et redirige les requêtes vers app1 ou app2. J'utilise l'algorithme round-robin pour distribuer la charge de façon équitable.

4 Services Node.js — MongoDB · Redis · RabbitMQ

4.1 Connexion MongoDB (app/services/mongodb.js):

Je définis le schéma d'une commande et j'établis la connexion à la base. Les données sont

persistantes grâce au volume Docker que j'ai configuré.

```
File  Edit  Selection  View  Go  ···  ←  →  Search
JS mongodb.js ×
C: > Users > pc > tp-archi-distribuee > app > services > JS mongodb.js > ...
1   const mongoose = require('mongoose');
2
3   async function connectMongo() {
4       try {
5           // Connexion à l'adresse du conteneur mongodb défini dans docker-compose
6           await mongoose.connect('mongodb://mongodb:27017/tp_archi');
7           console.log('✅ MongoDB connecté');
8       } catch (err) {
9           console.error('❌ Erreur MongoDB:', err.message);
10          process.exit(1);
11      }
12  }
13
14  const Commande = mongoose.model('Commande', {
15      produit: String,
16      quantite: Number,
17      client: String,
18      date: { type: Date, default: Date.now }
19  });
20
21  module.exports = { connectMongo, Commande };
```

4.2 Connexion Redis (app/services/redis.js):

Je crée une simple connexion au conteneur Redis pour stocker temporairement les résultats. Ça m'évite d'interroger MongoDB à chaque requête GET.

```
C: > Users > pc > tp-archi-distribuee > app > services > JS redis.js > ...
1   const Redis = require('ioredis');
2   💡
3   // Connexion au conteneur redis
4   const redis = new Redis({
5       host: 'redis',
6       port: 6379
7   });
8
9   redis.on('connect', () => console.log('✅ Redis connecté'));
10  redis.on('error', (err) => console.error('❌ Erreur Redis:', err));
11
12  module.exports = redis;
```

4.3 Connexion RabbitMQ (app/services/rabbitmq.js):

Je crée un canal de communication et une file d'attente durable. À chaque commande, je publie

un message qui sera consommé de façon asynchrone.

```
File  Edit  Selection  View  Go  ···  ←  →  Search

JS rabbitmq.js ×

C: > Users > pc > tp-archi-distribuee > app > services > JS rabbitmq.js > ...
```

```javascript
const amqp = require('amqplib');

let channel;

async function connectRabbitMQ() {
    try {
        const connection = await amqp.connect('amqp://admin:admin@rabbitmq');
        channel = await connection.createChannel();
        await channel.assertQueue('commandes', { durable: true });
        console.log('✅ RabbitMQ connecté');
    } catch (err) {
        console.error('❌ Erreur RabbitMQ:', err.message);
    }
}

function publierCommande(commande) {
    if (channel) {
        channel.sendToQueue('commandes', Buffer.from(JSON.stringify(commande)));
    }
}

module.exports = { connectRabbitMQ, publierCommande };
```

5 Routes API — POST /commande & GET /commandes

```javascript
// GET /commandes : Lister les commandes avec cache
router.get('/', async (req, res) => {
    try {
        const cached = await redis.get('toutes_commandes');
        if (cached) {
            return res.json({
                source: 'CACHE Redis',
                traitee_par: INSTANCE,
                commandes: JSON.parse(cached)
            });
        }

        const commandes = await Commande.find().sort({ date: -1 });
        await redis.setex('toutes_commandes', TTL, JSON.stringify(commandes));

        res.json({
            source: 'MongoDB',
            traitee_par: INSTANCE,
            commandes
        });
    } catch (err) {
        res.status(500).json({ erreur: err.message });
    }
});

module.exports = router;
```

Dans le POST, je sauvegarde en base, je vide le cache et je publie dans RabbitMQ. Pour le GET, je vérifie d'abord Redis avant d'aller chercher en MongoDB.

6 Serveur principal & Consommateur RabbitMQ

J'ai créé un microservice indépendant qui surveille la file RabbitMQ. Dès qu'un message arrive, il simule l'envoi d'un email de confirmation.

6.1 Serveur principal (app/server.js):

server.js

C: > Users > pc > tp-archi-distribuee > app > server.js > start

```javascript
const express = require('express');
const { connectMongo } = require('./services/mongodb');
const { connectRabbitMQ } = require('./services/rabbitmq');
const commandesRoutes = require('./routes/commandes');
const app = express();
const PORT = process.env.PORT || 3001;
const INSTANCE = process.env.INSTANCE || 'Instance-?';
app.use(express.json());
// Liaison des routes définies précédemment
app.use('/commandes', commandesRoutes);
// Route de santé pour vérifier si l'instance est active
app.get('/health', (req, res) => {
    res.json({ status: 'OK', instance: INSTANCE });
});
// Démarrage des connexions aux services puis du serveur
async function start() {
    try {
        await connectMongo();
        await connectRabbitMQ();
        app.listen(PORT, () => {
            console.log(`${INSTANCE} lancée sur le port ${PORT}`);
        });
    } catch (err) {
        console.error('Erreur au démarrage:', err);
        process.exit(1);
    }
}
start();
```

6.2 Consommateur RabbitMQ (app/consumer.js):

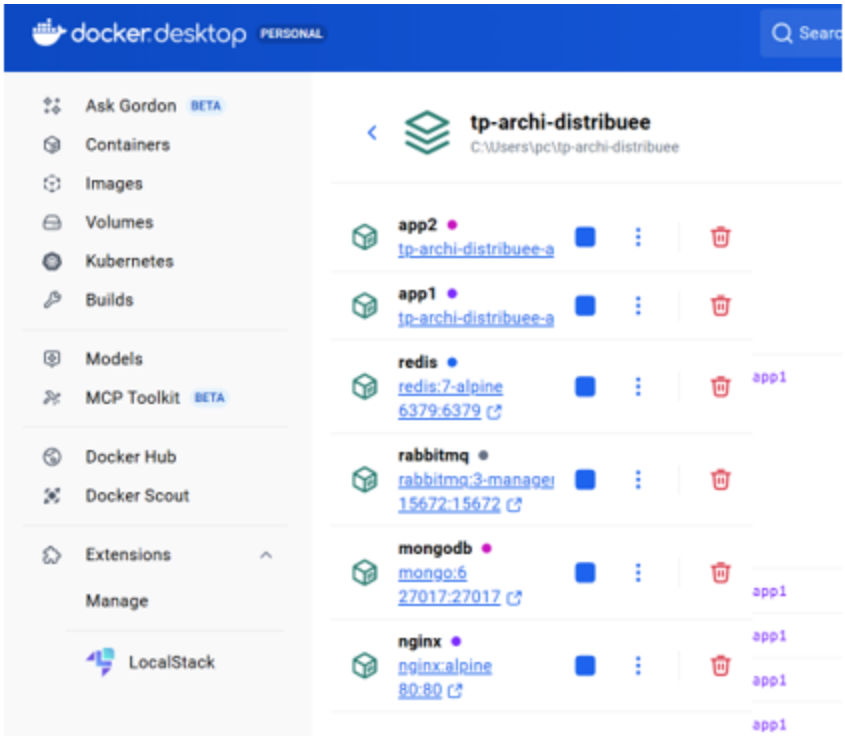
```
JS consumer.js ×
C: > Users > pc > tp-archi-distribuee > app > JS consumer.js > ...
const amqp = require('amqplib');

async function demarrerConsommateur() {
    try {
        const conn = await amqp.connect('amqp://admin:admin@rabbitmq');
        const channel = await conn.createChannel();
        const QUEUE = 'commandes';

        await channel.assertQueue(QUEUE, { durable: true });
        console.log('🔔 Consommateur en attente de messages...');

        channel.consume(QUEUE, (msg) => {
            if (msg) {
                const commande = JSON.parse(msg.content.toString());
                console.log('------------------------------');
                console.log('🚀 NOUVELLE COMMANDE REÇUE !');
                console.log('Client:', commande.client);
                console.log('Produit:', commande.produit);
                console.log('ID :', commande.id);
                console.log('📧 Confirmation simulée envoyée.');
                console.log('------------------------------');
                channel.ack(msg); // Accuser réception
            }
        });
    } catch (err) {
        console.log('En attente de RabbitMQ...');
        setTimeout(demarrerConsommateur, 5000); // Réessayer si RabbitMQ n'est pas prêt
    }
}
```

6.3 Dockerfile (app/Dockerfile):

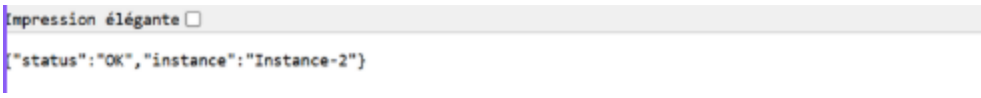
```
Dockerfile ×
C: > Users > pc > tp-archi-distribuee > app > Dockerfile > FROM
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
CMD ["node", "server.js"]
```

7 Lancement du système complet

7.1 Démarrer tous les conteneurs:



7.2 Vérifier que tout est actif:



8 Tests avec Postman — Scénario complet

GET http://localhost/commandes

Docs Params Authorization Headers 7 Body Scripts Tests Settings

Body Cookies Headers 7 Test Results

{} JSON Preview Visualize


```
1  {
2      "source": "MongoDB",
3      "traitee_par": "Instance-1",
4      "commandes": [
5          {
6              "_id": "69bedf2a70c9fd404fc00c1e",
7              "produit": "Laptop Dell XPS",
8              "quantite": 2,
9              "client": "Ahmed Benali",
10             "date": "2026-03-21T18:10:50.840Z",
11             "__v": 0
12         },
13         {
14             "_id": "69beddfde7ada447141abf8a",
15             "produit": "Laptop Dell XPS",
16             "quantite": 2,
17             "client": "Ahmed Benali",
18             "date": "2026-03-21T18:05:49.997Z",
19             "__v": 0
20         }
21     ]
22 }
```


GET http://localhost/commandes

Docs Params Authorization Headers 7 Body Scripts Tests Settings

Body Cookies Headers 7 Test Results 200 OK

{} JSON Preview Visualize


```
1  {
2      "source": "CACHE Redis",
3      "traitee_par": "Instance-2",
4      "commandes": [
5          {
6              "_id": "69bedf2a70c9fd404fc00c1e",
7              "produit": "Laptop Dell XPS",
8              "quantite": 2,
9              "client": "Ahmed Benali",
10             "date": "2026-03-21T18:10:50.840Z",
11             "__v": 0
12         },
13         {
14             "_id": "69beddfde7ada447141abf8a",
15             "produit": "Laptop Dell XPS",
16             "quantite": 2,
17             "client": "Ahmed Benali",
18             "date": "2026-03-21T18:05:49.997Z",
19             "__v": 0
20         }
21     ]
22 }
```


Résumé

Ce TP m'a permis de mettre en place une architecture distribuée avec 6 composants qui communiquent entre eux. Nginx gère la répartition de charge, Node.js traite la logique métier, MongoDB assure la persistance, Redis accélère les lectures répétitives et RabbitMQ permet un traitement asynchrone des événements. Ce qu'on retient surtout c'est comment chaque brique a un rôle précis, et que la combinaison de tout ça rend le système plus robuste et scalable qu'une appli classique monolithique.

Questions de compte-rendu, à rendre avec vos captures Postman

Q1	**Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?**
Q2	**Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ? Pourquoi ?**
Q3	**Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET (source Redis).**
Q4	**Pourquoi invalide-t-on le cache Redis après un POST /commande ?**
Q5	**En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?**
Q6	**Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?**

Q1: C'est **Nginx** qui reçoit les requêtes en premier. Il utilise l'algorithme **round-robin** : il alterne les requêtes entre app1 et app2.

Q2: Nginx continue à fonctionner et redirige toutes les requêtes vers app2. C'est justement l'intérêt du load balancer : il détecte qu'app1 ne répond plus et l'écarte automatiquement.

Q3: Le 1er GET interroge directement MongoDB (plus lent), puis stocke le résultat dans Redis. Le 2ème GET récupère la réponse depuis Redis (quasi instantané), sans toucher la base.

Q4: Après un POST, les données ont changé en base. Si on garde l'ancien cache, le GET retournerait des données obsolètes. On le supprime pour forcer une resynchronisation avec MongoDB.

Q5: RabbitMQ découple le traitement des commandes : si le consommateur tombe, les messages restent dans la file et seront traités plus tard. Le système principal n'est pas bloqué.

Q6: Il faudrait ajouter une ligne dans le bloc `upstream backend` dans `nginx.conf` :

```
server app3:3003;
```